

CS3236

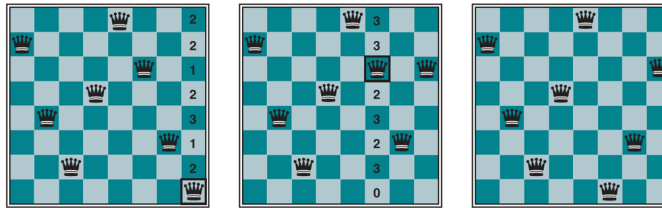
AY24/25 SEM 1

yycbb

- Conflict set of X: set of assignments which shrink the domain of X
- Backjumping: backtrack to the most recent assignment in the conflict set
- Min-Conflict Local Search

function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or *failure*
inputs: *csp*, a constraint satisfaction problem
max_steps, the number of steps allowed before giving up

current $\leftarrow$ an initial complete assignment for *csp*
for *i* = 1 to *max_steps* **do**
 if *current* is a solution for *csp* **then return** *current*
 var $\leftarrow$ a randomly chosen conflicted variable from *csp*.VARIABLES
 value $\leftarrow$ the value *v* for *var* that minimizes CONFLICTS(*csp*, *var*, *v*, *current*)
 set *var* = *value* in *current*
return *failure*



Propositional Logic

- Satisfiability** $\rightarrow$ If α is true in *m*, *m* satisfies α and *m* is a model of α . $M(\alpha)$ denotes the set of all models of α .
- Entailment** $\rightarrow \alpha \models \beta$ iff in every model in which α is true, β is also true. $M(\alpha) \subseteq M(\beta)$

Sentence $\rightarrow$ *AtomicSentence* | *ComplexSentence*
AtomicSentence $\rightarrow$ True | False | *P* | *Q* | *R* | ...
ComplexSentence $\rightarrow$ (*Sentence*)
| $\neg$ *Sentence*
| *Sentence* $\wedge$ *Sentence*
| *Sentence* $\vee$ *Sentence*
| *Sentence* $\Rightarrow$ *Sentence*
| *Sentence* $\Leftrightarrow$ *Sentence*

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Figure 7.7 A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

- Model Checking:** Check for all models of KB (when KB is True)
- Theorem Proving:**
 - $\alpha \models \beta$ iff $\alpha \Rightarrow \beta$ iff $\alpha \wedge \neg\beta$ is unsat

Inference Rule	Name
$A, B \models A \wedge B$	Combination rule CR
$A \wedge B \models A$	Simplification rule SR
$A \models A \vee B$	Addition rule AR
$A, A \Rightarrow B \models B$	Modus ponens MP
$\neg B, A \Rightarrow B \models \neg A$	Modus tollens MT
$A \Rightarrow B, B \Rightarrow C \models A \Rightarrow C$	Hypothetical syllogism HS
$A \vee B, \neg A \models B$	Disjunctive syllogism DS
$A \Rightarrow B, \neg A \Rightarrow B \models B$	Rule of cases RC
$A \Leftrightarrow B \models A \Rightarrow B$	Equivalence elimination EE
$A \Rightarrow B, B \Rightarrow A \models A \Leftrightarrow B$	Equivalence introduction EI
$A, \neg A \models B$	Inconsistency rule IR
$A \wedge B \models B \wedge A$	“and” Commutivity rule ACR
$A \vee B \models B \vee A$	“or” Commutivity rule OCR
If $A_1, A_2, \dots, A_n, B \models C$ then $A_1, A_2, \dots, A_n \models B \Rightarrow C$	Deduction theorem DT

- Resolution:**
 - Only applies to CNF
 - $(A \vee P), (B \vee \neg P) \models A \vee B$
 - If A and B have copies of the same literal, remove one.
 - Algorithm:

function PL-RESOLUTION(*KB*, α) **returns** *true* or *false*
inputs: *KB*, the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

clauses $\leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$
new $\leftarrow \{\}$
while *true* **do**
 for each pair of clauses C_i, C_j **in** *clauses* **do**
 resolvents $\leftarrow$ PL-RESOLVE(C_i, C_j)
 if *resolvents* contains the empty clause **then return** *true*
 new $\leftarrow$ *new* $\cup$ *resolvents*
 if *new* $\subseteq$ *clauses* **then return** *false*
 clauses $\leftarrow$ *clauses* $\cup$ *new*

- Definite Clause** $\rightarrow$ disjunction of literals where exactly one is positive.
- Horn Clause** $\rightarrow$ disjunction of literals where at most one is positive.
Deciding entailment with pure horn clause KB is linear-time.

CSP

- AC3 (Arc Consistency)

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise
queue $\leftarrow$ a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**
 $(X_i, X_j) \leftarrow$ POP(*queue*)
 if REVISE(*csp*, X_i, X_j) **then**
 if size of $D_i = 0$ **then return** *false*
 for each X_k **in** X_i .NEIGHBORS - $\{X_j\}$ **do**
 add (X_k, X_i) to *queue*
 return *true*

function REVISE(*csp*, X_i, X_j) **returns** true iff we revise the domain of X_i
revised $\leftarrow$ *false*
for each *x* in D_i **do**
 if no value *y* in D_j allows (x, y) to satisfy the constraint between X_i and X_j **then**
 delete *x* from D_i
 revised $\leftarrow$ *true*
return *revised*

- Backtracking Search

function BACKTRACKING-SEARCH(*csp*) **returns** a solution or *failure*
return BACKTRACK(*csp*, $\{\}$)

function BACKTRACK(*csp*, *assignment*) **returns** a solution or *failure*
if *assignment* is complete **then return** *assignment*
var $\leftarrow$ SELECT-UNASSIGNED-VARIABLE(*csp*, *assignment*)
for each *value* in ORDER-DOMAIN-VALUES(*csp*, *var*, *assignment*) **do**
 if *value* is consistent with *assignment* **then**
 add $\{var = value\}$ to *assignment*
 inferences $\leftarrow$ INFERENCE(*csp*, *var*, *assignment*)
 if *inferences* $\neq$ *failure* **then**
 add *inferences* to *csp*
 result $\leftarrow$ BACKTRACK(*csp*, *assignment*)
 if *result* $\neq$ *failure* **then return** *result*
 remove *inferences* from *csp*
 remove $\{var = value\}$ from *assignment*
return *failure*

Improving Backtracking

- Variable Ordering (SELECT-UNASSIGNED-VARIABLE):
 - minimum remaining value heuristic
 - degree heuristic: choose one with largest number of constraints on other unassigned variables
- Value Ordering (ORDER-DOMAIN-VALUES): least constraining value heuristic
- Interleaving Search and Inference:
 - Forward Checking: only enforce arc consistency among immediate neighbours
 - MAC: resursively propagates constraints
- Conflict Set Backjumping:

First Order Logic

<i>Sentence</i>	→	<i>AtomicSentence</i> <i>ComplexSentence</i>
<i>AtomicSentence</i>	→	<i>Predicate</i> <i>Predicate</i> (<i>Term</i> ,...) <i>Term</i> = <i>Term</i>
<i>ComplexSentence</i>	→	(<i>Sentence</i>)
		¬ <i>Sentence</i>
		<i>Sentence</i> ∧ <i>Sentence</i>
		<i>Sentence</i> ∨ <i>Sentence</i>
		<i>Sentence</i> ⇒ <i>Sentence</i>
		<i>Sentence</i> ⇔ <i>Sentence</i>
		<i>Quantifier Variable</i> ,... <i>Sentence</i>
<i>Term</i>	→	<i>Function</i> (<i>Term</i> ,...)
		<i>Constant</i>
		<i>Variable</i>
<i>Quantifier</i>	→	∀ ∃
<i>Constant</i>	→	<i>A</i> <i>X</i> ₁ <i>John</i> ...
<i>Variable</i>	→	<i>a</i> <i>x</i> <i>s</i> ...
<i>Predicate</i>	→	<i>True</i> <i>False</i> <i>After</i> <i>Loves</i> <i>Raining</i> ...
<i>Function</i>	→	<i>Mother</i> <i>LeftLeg</i> ...
OPERATOR PRECEDENCE	:	¬, =, ∧, ∨, ⇒, ⇔

Figure 8.3 The syntax of first-order logic with equality, specified in Backus–Naur form (see page 1030 if you are not familiar with this notation). Operator precedences are specified, from highest to lowest. The precedence of quantifiers is such that a quantifier holds over everything to the right of it.

- **Universal Instantiation:** $\frac{\forall v \alpha}{SUBST(v/g, \alpha)}$
- **Existential Instantiation:** same as UI with g replaced by k, a Skolem constant that does not appear elsewhere in the KB
- Caveat for reusing the symbol: UI before EI then it is logically wrong.
- **Unification:**
 - a variable cannot be replaced by a term containing itself (e.g. a function of itself)
 - Most General Unifier (MGU)

Algorithm 3.1 Unification

Input: Two sentences *A* and *B*; an empty set of substitutions θ .
Output: A most general unifier of the sentences if they can be unified; otherwise failure.

Procedure *unify*(*A*, *B*, **var** θ);
 scan *A* and *B* from left to right
 until *A* and *B* disagree on a symbol or *A* and *B* are exhausted;
 if *A* and *B* are not exhausted
 let *x* and *y* be the symbols where *A* and *B* disagree;
 if *x* is a variable
 $\theta = \theta \cup \{x/y\}$;
 unify(*subst*(θ , *A*), *subst*(θ , *B*), θ);
 else if *y* is a variable
 $\theta = \theta \cup \{y/x\}$;
 unify(*subst*(θ , *A*), *subst*(θ , *B*), θ);
 else
 $\theta = \text{Failure}$;
 endif
 endif

- **end if**
- *occur-check needed before each update of θ
-

• **Forward Chaining:**

function FOL-FC-ASK(*KB*, α) **returns** a substitution or *false*
 inputs: *KB*, the knowledge base, a set of first-order definite clauses
 α , the query, an atomic sentence

while *true* **do**
 new ← { } // The set of new sentences inferred on each iteration
 for each rule in *KB* **do**
 ($p_1 \wedge \dots \wedge p_n \Rightarrow q$) ← STANDARDIZE-VARIABLES(*rule*)
 for each θ such that SUBST(θ , $p_1 \wedge \dots \wedge p_n$) = SUBST(θ , $p'_1 \wedge \dots \wedge p'_n$)
 for some $p'_1, \dots, p'_n$ in *KB*
 $q' \leftarrow \text{SUBST}(\theta, q)$
 if q' does not unify with some sentence already in *KB* or *new* **then**
 add q' to *new*
 $\phi \leftarrow \text{UNIFY}(q', \alpha)$
 if ϕ is not *failure* **then return** ϕ
 if *new* = { } **then return** *false*
 add *new* to *KB*

-

• **Backward Chaining:**

function FOL-BC-ASK(*KB*, *query*) **returns** a generator of substitutions
 return FOL-BC-OR(*KB*, *query*, { })

function FOL-BC-OR(*KB*, *goal*, θ) **returns** a substitution
 for each rule in FETCH-RULES-FOR-GOAL(*KB*, *goal*) **do**
 (*lhs* ⇒ *rhs*) ← STANDARDIZE-VARIABLES(*rule*)
 for each θ' in FOL-BC-AND(*KB*, *lhs*, UNIFY(*rhs*, *goal*, θ)) **do**
 yield θ'

function FOL-BC-AND(*KB*, *goals*, θ) **returns** a substitution
 if $\theta = \text{failure}$ **then return**
 else if LENGTH(*goals*) = 0 **then yield** θ
 else
 first, *rest* ← FIRST(*goals*), REST(*goals*)
 for each θ' in FOL-BC-OR(*KB*, SUBST(θ , *first*), θ) **do**
 for each θ'' in FOL-BC-AND(*KB*, *rest*, θ') **do**
 yield θ''

Figure 9.6 A simple backward-chaining algorithm for first-order knowledge bases.

-
- **Resolution:**

- Convert to CNF:
 - Use implication rule
 - Move $\neg$ inside
 - Standardise variables: replace repeated variables
 - Skolemise: translate $\forall x \exists y P(y)$ into $P(F(x))$
 - Drop universal quantifiers
 - Distribute disjunction over conjunction

BN

$(X \perp Y | Z)$

$(X \perp Y | Z)$

$(X \perp Y)$

$$P(y|x, z) = \frac{P(x, y, z)}{P(x, z)} = \frac{P(x)P(y)P(z|x, y)}{P(x)P(z|x, y)} = P(y|z)$$

$$P(x, y|z) = \frac{P(x, y, z)}{P(z)} = \frac{P(x)P(y)P(z|x, y)}{P(z)} = P(x|z)P(y|z)$$

- $P(x, y, z) = P(x)P(y)P(z|x, y)$
- Summing over z , we see that
 - $P(x, y) = P(x)P(y)$
 - i.e. x and y are marginally independent.

1. Consider just the **ancestral subgraph** consisting of **X, Y, Z**, and their ancestors.
2. Add links between any unlinked pair of nodes that share a common child; this is the **moral graph**.
3. Replace all directed links by undirected links.
4. If **Z** blocks all paths between **X** and **Y** in the resulting graph, then **Z** d-separates **X** and **Y**.
 - In that case, X is conditionally independent of Y , given Z .
 - Otherwise, the original Bayes net does not imply conditional independence.

The Markov Blanket for X consists of the set of parents of X, the set of children of X, and the set of parents of children of X.